

Hardware Bring-Up

Follow this in order. Each stage ends with a **checkpoint** — if it fails, stop and fix it there. Skipping ahead means debugging two unknowns at once, which is what turns an afternoon into a week.

Realistic timing: **Stages 0–3 take a day**. Stages 4–7 take another. Most of the pain is in Stage 3, and almost none of it is the code.

Stage 0 — Before you touch anything

You need:

- 6 × Raspberry Pi 4 with Raspberry Pi OS flashed, SSH enabled, on your Wi-Fi
- 6 × WaveShare SX1262 868M LoRa HAT with SMA antennas
- Internet on every Pi (`setup.sh` downloads WaveShare's driver)
- A laptop on the same network

Label the Pis physically now. Write on tape: `GW`, `CH-A`, `2`, `3`, `CH-B`, `5`. You will otherwise lose ten minutes every time you need to know which is which.

Label	LoRa addr	Role	Notes
GW	0	Gateway	Needs internet. Runs the dashboard
CH-A	1	Cluster head A	Must reach the gateway
2	2	Node, cluster A	
3	3	Node, cluster A, backup CH-A	
CH-B	4	Cluster head B	Must NOT reach the gateway
5	5	Node, cluster B, backup CH-B	Must reach CH-B

⚠ **Screw the antenna on before powering any HAT.** Transmitting without an antenna can destroy the power amplifier.

Stage 1 — Jumpers (physical, all six HATs)

This is the single most common cause of "the radio doesn't work".

1. **UART selection jumper** → **position B**. Position B connects the LoRa module to the Raspberry Pi. A is USB, C is a pass-through.
2. **Remove the M0 and M1 jumper caps entirely.** Put them somewhere safe. The driver controls the module's power mode by driving these pins from BCM22 and BCM27. **A pin reads HIGH when its cap is removed.** With the caps fitted, the Pi physically cannot change the radio's mode — you lose all power saving, and the module may sit in the wrong mode entirely.

3. **Antenna screwed on**, finger-tight, before any power is applied.

Checkpoint: all six HATs have jumper on B, no M0/M1 caps, antenna attached.

Stage 2 — Provision one Pi (start with the gateway)

SSH into the gateway Pi.

```
sudo apt update && sudo apt install -y git
git clone https://github.com/nishankswamy/forest-fire-dashboard.git ~/forest-fire-dashboard
cd ~/forest-fire-dashboard/pi
sudo ./setup.sh gateway
```

What it does: installs Python packages, downloads WaveShare's `sx126x.py`, enables UART, **disables the serial login console**, strips `console=serial0` from the kernel command line, adds you to `dialout`, validates the TDMA schedule, and installs the systemd services.

Those two console steps matter more than they look — the login console holds `/dev/ttyS0` open and fights the HAT for it. The symptom is a radio that receives nothing, which sends people hunting for hardware faults.

```
sudo reboot
```

After it comes back:

```
ls -l /dev/serial*
python3 -c "import sx126x; print('driver ok')"
```

Checkpoint: `serial0 -> ttyS0` and the driver imports.

If `serial0` points at `ttYAMA0` instead, every later command needs `LORA_SERIAL=/dev/ttyAMA0` in front of it, and the systemd units need `Environment="LORA_SERIAL=/dev/ttyAMA0"` adding.

Now do exactly the same on **CH-A**, but with:

```
sudo ./setup.sh node 1
```

Stop the services on both for now — Stage 3 needs the radio to itself:

```
# on GW
sudo systemctl stop fire-gateway fire-api
# on CH-A
sudo systemctl stop fire-node
```

Stage 3 — Prove the radio between two Pis

Do not skip this. It tests the physical link with no protocol on top, so a failure here has exactly one cause.

Put both Pis on a desk, a couple of metres apart.

On the gateway:

```
cd ~/forest-fire-dashboard/pi/tools
python3 linktest.py rx
```

On CH-A:

```
cd ~/forest-fire-dashboard/pi/tools
python3 linktest.py tx --node-id 1 --dst 0 --count 50
```

You should see packets appearing on the gateway as they are sent. Then Ctrl-C the receiver for a scored summary:

```
node      recv    lost  delivered  crc_fail  rssi_mean
1          50      0    100.0%      0         -61.4
```

Checkpoint: `delivered ≥ 99%`, `crc_fail 0`, `rssi` better than `-100 dBm`.

If it says NOTHING RECEIVED

Work down this list in order — it is ordered by how often each is the cause:

Check	How
Jumper on B, M0/M1 caps removed	Look at the board
Both Pis on the same settings	<code>FREQ</code> , <code>AIR_SPEED</code> , <code>NET_ID</code> , <code>CRYPT_KEY</code> in <code>pi/common/config.py</code> must be identical. They are, unless you edited one
Wrong serial device	<code>ls -l /dev/serial*</code> — if not <code>ttyS0</code> , set <code>LORA_SERIAL</code>
Serial console still holding the port	<code>systemctl status serial-getty@ttyS0</code> — should be inactive/disabled
Antenna not actually screwed down	Finger-tight, both ends
A service still running and using the radio	<code>sudo systemctl stop fire-node fire-gateway</code>

Now do the placement survey

This determines where the Pis physically go, and the design depends on it.

Test each of these links with `linktest.py`, moving Pis until each behaves:

Link	Required
GW ↔ CH-A	must work
CH-A ↔ node 2	must work
CH-A ↔ node 3	must work
CH-A ↔ CH-B	must work
CH-B ↔ node 5	must work
GW ↔ CH-B	must FAIL

That last row is not a mistake. CH-B being out of gateway range is what forces the three-hop relay. If CH-B can reach the gateway directly, move it further away or put a building between them — otherwise the multi-hop path never exercises and there is nothing to demonstrate.

Indoors, "out of range" usually means a couple of floors or a few concrete walls. Outdoors it may mean several hundred metres.

Checkpoint: all five required links pass; GW↔CH-B fails.

Stage 4 — Two Pis on the real firmware

Still just the gateway and CH-A.

On the gateway, two SSH sessions:

```
# session 1
cd ~/forest-fire-dashboard/pi/gateway
python3 gateway.py
```

```
# session 2
cd ~/forest-fire-dashboard/pi/gateway
python3 api.py
```

On CH-A:

```
cd ~/forest-fire-dashboard/pi/node
NODE_ID=1 SMOKE_MODE=simulate DHT_MODE=simulate python3 sensor_node.py
```

Running by hand rather than via systemd means you see the logs directly.

Within about a minute the gateway should print something like:

```
[gw] CH-A — Ridge East      seq=  0 24.4C 55%RH 60ppm batt 95% via 1 (1 hop) rssi -62dBm
NORMAL
```

And CH-A should show it received a beacon and sent a reading.

Checkpoint: the gateway logs a reading from node 1, roughly every 60 s.

Nothing arrives, but linktest worked? Then it is timing, not radio. Check CH-A's log for `no beacon for N frames — free-running`. If it never hears the beacon, it transmits on its own clock and lands outside its slot.

Stage 5 — Bring up the remaining four

Provision each, one at a time, checking after each addition rather than all at once:

```
# on node 2
sudo ./setup.sh node 2
# on node 3
sudo ./setup.sh node 3
# on CH-B
sudo ./setup.sh node 4
# on node 5
sudo ./setup.sh node 5
```

Then switch everything to run as services so it survives reboots:

```
# gateway
sudo systemctl start fire-gateway fire-api

# each node
sudo systemctl start fire-node
```

Watch the gateway:

```
journalctl -u fire-gateway -f
```

Within two frames (about two minutes) you should see all five nodes. Look at the **hop counts** — they are the proof that routing works:

```
[gw] CH-A — Ridge East      ... via 1 (1 hop)  NORMAL
[gw] Node 2 — Fire Line A    ... via 1 (2 hops) NORMAL
[gw] Node 3 — Watchtower     ... via 1 (2 hops) NORMAL
[gw] CH-B — Creek Bed        ... via 1 (2 hops) NORMAL
[gw] Node 5 — Bamboo Belt    ... via 1 (3 hops) NORMAL
```

Checkpoint: all five nodes reporting, and **node 5 shows 3 hops**.

If node 5 shows 2 hops, it reached CH-A directly — move it so only CH-B can hear it.

Stage 6 — Live dashboard

From any device on the same network:

```
http://<gateway-pi-ip>:5000
```

Find the IP with `hostname -I` on the gateway.

No file editing is needed. The dashboard probes `/api/site` on load: served by the gateway it uses live data, served anywhere else it falls back to the simulator. The footer tells you which. Force either with `?sim` or `?live`.

What to verify:

1. **Five markers plus a blue GW** — CH-A and CH-B drawn larger with rings
2. **Click node 5** — the routing panel shows `Hops to gateway: 3` and the path `N-05 → N-04 → N-01 → GW`, drawn on the map
3. **Network Protocol panel** shows 2 cluster heads, 2 backups
4. **Readings updating** roughly every minute

Checkpoint: node 5's three-hop path draws on the map from real packets.

Prove the alert path

Temporarily lower the thresholds so ordinary room conditions trip it:

```
# on one node, stop the service and run by hand
sudo systemctl stop fire-node
cd ~/forest-fire-dashboard/pi/node
NODE_ID=2 SMOKE_MODE=simulate DHT_MODE=simulate \
python3 -c "
import sys; sys.path.insert(0,'../common')
import config
config.RULES['temp_high'] = 20; config.RULES['smoke_high'] = 10
config.RULES['hum_low'] = 99
import sensor_node; sensor_node.main()
"
```

After two cycles the banner should go red and name the node.

Restore it afterwards — `sudo systemctl start fire-node`.

Test the remote controls

In the Network Protocol panel: **Stop** halts the whole network, **Restart** resets it. Both travel in the beacon, so they take one frame to reach cluster A and two to reach cluster B. A halt auto-expires after 30 minutes.

Stage 7 — Real sensors

Only now. The full pin map is below — the same diagram as `pi/docs/wiring.svg`.

Sensor node wiring – Raspberry Pi 4 + SX1262 LoRa HAT

One of five identical sensor nodes. The gateway Pi uses the HAT only – no DHT22, MQ-2 or MCP3008.

5V 3V3 GND LoRa HAT DHT22 MQ-2 MCP3008

40-pin header (pins 1–26 shown)

3V3	1	2	5V
GPI02 SDA	3	4	5V
GPI03 SCL	5	6	GND
GPI04	7	8	GPI014 TXD
GND	9	10	GPI015 RXD
GPI017	11	12	GPI018
GPI027	13	14	GND
GPI022	15	16	GPI023
3V3	17	18	GPI024
GPI010 MOSI	19	20	GND
GPI09 MISO	21	22	GPI025
GPI011 SCLK	23	24	GPI08 CE0
GND	25	26	GPI07

Pins 27–40 unused.

A stacking header is needed to reach these under the HAT.

SX1262 868M LoRa HAT (SKU 16806)

Sits directly on the header. Point-to-point, NOT LoRaWAN.

UART TXD	pin 8	GPI014
UART RXD	pin 10	GPI015
M1	pin 13	GPI027
M0	pin 15	GPI022

Jumper to position B. REMOVE the M0/M1 caps. Antenna on before power.

DHT22 – temperature + humidity

DHT_MODE=dht22

VCC	pin 1	3V3
DATA	pin 7	GPI04
GND	pin 9	GND

10 kΩ pull-up from DATA to 3V3. Occasional None reads are normal.

MQ-2 – smoke / combustible gas

SMOKE_MODE=digital (D0) or SMOKE_MODE=adc (A0)

VCC	pin 2	5V
D0 (digital mode)	pin 11	GPI017
A0 (adc mode)		MCP3008 CH0

GND to any GND pin. Set the D0 trip point with the on-board pot.

MCP3008 – 10-bit ADC on SPI

SMOKE_MODE=adc only. Enable SPI: raspi-config -> Interface Options -> SPI

VDD + VREF	pin 17	3V3
AGND + DGND		GND
CLK	pin 23	GPI011 SCLK
DOUT	pin 21	GPI09 MISO
DIN	pin 19	GPI010 MOSI
CS	pin 24	GPI08 CE0
CH0 / CH1		MQ-2 A0 / battery divider

! 5V hazard: the MQ-2 runs on 5V and its A0 can exceed 3.3V, which will damage the MCP3008. Fit a 2x10 kΩ divider between A0 and CH0.

Per sensor node:

DHT22	Pi
VCC	3.3 V (pin 1)
DATA	BCM 4 (pin 7), 10 kΩ pull-up to 3.3 V
GND	GND (pin 9)

MCP3008	Pi
VDD, VREF	3.3 V (pin 17)
AGND, DGND	GND
CLK	BCM 11 / SCLK (pin 23)
DOUT	BCM 9 / MISO (pin 21)
DIN	BCM 10 / MOSI (pin 19)
CS	BCM 8 / CE0 (pin 24)
CH0	MQ-2 AO through a 2 × 10 kΩ divider

MQ-2	Pi
VCC	5 V (pin 2)
GND	GND
AO	MCP3008 CH0, via the divider

⚠ **The divider is mandatory.** The MQ-2 runs on 5 V and its analogue output can exceed 3.3 V, which will destroy the MCP3008. Two 10 kΩ resistors between AO and CH0, tap in the middle.

You will need a **stacking header** to reach the GPIO pins under the HAT.

Then switch the node off simulated readings:

```
sudo nano /etc/systemd/system/fire-node.service
# SMOKE_MODE=simulate -> SMOKE_MODE=adc
# DHT_MODE=simulate -> DHT_MODE=dht22
sudo systemctl daemon-reload && sudo systemctl restart fire-node
journalctl -u fire-node -f
```

Checkpoint: real temperature and humidity in the log, tracking the room.

The MQ-2 needs 24–48 hours powered to burn in before its readings settle, and the ppm figure is an estimate from a resistance curve, not a calibrated measurement. Say so in the report rather than claiming absolute accuracy.

Daily operation

```
# is everything alive?
systemctl status fire-gateway fire-api      # gateway
systemctl status fire-node                  # any node

# watch traffic
journalctl -u fire-gateway -f

# quick health check
curl http://localhost:5000/api/health
curl http://localhost:5000/api/routes      # how packets ACTUALLY arrived

# push a code change from your laptop, then on each Pi
cd ~/forest-fire-dashboard && git pull && sudo systemctl restart fire-node
```

`/api/routes` is the most useful one on the bench — it shows the route each node's traffic really took over the last hour, which is how you confirm CH-B is relaying rather than sneaking a direct link.

Troubleshooting index

Symptom	Most likely cause
setting fail, setting again at startup	Jumper not on B, or M0/M1 caps still fitted
Nothing received at all	Work through the Stage 3 table
<code>linktest</code> works, firmware does not	Node never heard a beacon — check its log for <code>free-running</code>
Node 5 reports 2 hops instead of 3	It can hear CH-A directly. Move it
All nodes show offline on the dashboard	<code>fire-gateway</code> not running, or <code>OFFLINE_AFTER</code> shorter than your frame
Dashboard shows 50 nodes	It is on the simulator — you are not being served by the gateway. Check the footer
<code>[radio] WARNING: cannot control M0/M1</code>	Radio sleep is inactive. Everything works, but the duty-cycle figures do not hold
DHT22 returns <code>None</code> often	Normal for the part. The code retries and reuses the last value
Short range	Antenna near metal or at ground level. Raise it, keep <code>AIR_SPEED</code> at 2400
Cluster B silent after killing CH-A	Correct. CH-B can only hear CH-A and node 5, so it is genuinely partitioned and buffers

Before you call it working

- ☐ All five nodes reporting for a continuous hour

- ☐ Node 5 consistently at 3 hops in `/api/routes`
- ☐ `curl /api/health` shows `lastHour` climbing steadily
- ☐ Fire alert triggered and cleared at least once
- ☐ A node stopped and restarted, and the dashboard reflected both
- ☐ RSSI recorded for every link — this fills the report's measurement table